

Product Quantization, Worked Out by Hand

How a vector database stores a 32-byte embedding in 2 bytes and still ranks neighbours well — splitting the vector into sub-vectors and replacing each with a nearby centroid’s id — with the compression arithmetic and a distance lookup done by hand.

What this document does. It takes an 8-dimensional vector, splits it into sub-vectors, snaps each to the nearest centroid in a small codebook, and stores the resulting *codes* instead of the floats. It computes the exact compression ratio, shows the asymmetric-distance lookup table that makes search fast, and names the accuracy cost. Numbers from Appendix A.

Companion: scalar & binary quantization (#11) are the simpler cousins; recall@k (#12) is how you measure the accuracy you trade away.

1 The idea: store an id, not a vector

A single `float32` embedding of dimension $D = 768$ costs $768 \times 4 = 3072$ bytes. A billion of them is 3 TB — too much for RAM, and RAM is where fast search must live. Product Quantization (PQ) shrinks each vector by an order of magnitude or two by replacing chunks of it with *pointers into a small codebook*.

Intuition

Think of a paint catalogue. Instead of storing the exact RGB of every wall in a city, you store, per wall, the *catalogue number* of the closest standard colour. A few hundred catalogue colours cover everything “closely enough,” and a number is far smaller than a precise RGB triple. PQ does this per *slice* of the vector: divide the embedding into m slices, keep a 256-colour catalogue for each slice, and store m small ids. Reconstruction is approximate — but for *ranking* neighbours, close enough wins.

2 The construction

Split $\mathbf{v} \in \mathbb{R}^D$ into m contiguous sub-vectors of dimension D/m . For each sub-space, k -means has pre-computed a codebook of k centroids $\{c_1, \dots, c_k\}$. The PQ code replaces each sub-vector with the *index* of its nearest centroid:

$$\mathbf{v} \mapsto (i_1, \dots, i_m), \quad i_j = \arg \min_{1 \leq t \leq k} \|\mathbf{v}^{(j)} - c_t^{(j)}\|^2$$

Storage per vector is $m \lceil \log_2 k \rceil$ bits — with $k = 256$, exactly m bytes, *independent of D ’s precision*.

Compression, exactly. For $D = 8$ floats with $m = 2$ sub-spaces and $k = 256$:

$$\text{original} = 8 \times 32 = 256 \text{ bits} = 32 \text{ bytes},$$

$$\text{PQ code} = m \log_2 k = 2 \times 8 = 16 \text{ bits} = 2 \text{ bytes} \quad \Rightarrow \quad \boxed{16\times} \text{ smaller.}$$

At $D = 768$, $m = 96$: 3072 bytes $\rightarrow$ 96 bytes, a $32\times$ reduction — the ratio scales with the precision you give up, not with D .

Mini-example • encoding one vector

$\mathbf{v} = (1, 2, 0.5, 1.5, 3, 1, 2, 0)$, split into $\mathbf{v}^{(1)} = (1, 2, 0.5, 1.5)$ and $\mathbf{v}^{(2)} = (3, 1, 2, 0)$. With tiny $k = 4$ codebooks, the nearest centroids are

$$\mathbf{v}^{(1)} \rightarrow c_2^{(1)} = (1, 2, 0.5, 1.5) \text{ (index 2)}, \quad \mathbf{v}^{(2)} \rightarrow c_0^{(2)} = (3, 1, 2, 0) \text{ (index 0)}.$$

The whole vector is now the code $(2, 0)$ — two integers. To approximately reconstruct $\mathbf{v}$, look up $c_2^{(1)}$ and $c_0^{(2)}$ and concatenate.

3 Why search stays fast: the distance table

The point of PQ is not just storage — it is *cheap distance*. To compare a query $\mathbf{q}$ to millions of PQ codes, precompute, *once per query*, a table of the distance from each query sub-vector to every centroid:

$$\text{LUT}[j][t] = \|\mathbf{q}^{(j)} - \mathbf{c}_t^{(j)}\|^2, \quad j = 1 \dots m, \quad t = 1 \dots k.$$

Then the (squared) distance to any stored code $(i_1, \dots, i_m)$ is just m table look-ups summed:

$$\hat{d}(\mathbf{q}, \mathbf{v})^2 = \sum_{j=1}^m \text{LUT}[j][i_j].$$

No multiplications during the scan — m additions per candidate. This is *asymmetric* distance computation (ADC): the query stays full-precision, only the database is quantized, which keeps accuracy high.

Intuition

The cost moved. You pay $O(kD)$ once to build the table per query, then each of the N candidates costs only m look-ups instead of D multiply-adds. For $m \ll D$ and large N , that is a massive win — and it composes with HNSW (#9): use the graph to *find* candidates, use PQ to *score* them cheaply.

4 The cost you are paying

PQ is lossy. Two distinct vectors that fall in the same centroid become indistinguishable, so PQ distances are *estimates*: the ranking is approximately, not exactly, preserved. Mitigations: more centroids k (finer catalogue, more bits), more sub-spaces m (less averaging within a slice), or a re-ranking pass that recomputes *exact* distances on the top few hundred PQ candidates. The standard production recipe is HNSW + PQ for the shortlist, then exact re-rank — fast and accurate. How much accuracy you actually keep is measured by recall@k (#12).

Equation cheat-sheet

$$\text{code } (i_1, \dots, i_m), \quad i_j = \arg \min_t \|\mathbf{v}^{(j)} - \mathbf{c}_t^{(j)}\|^2, \quad \text{bits} = m \lceil \log_2 k \rceil$$

$$\hat{d}(\mathbf{q}, \mathbf{v})^2 = \sum_{j=1}^m \text{LUT}[j][i_j], \quad \text{LUT}[j][t] = \|\mathbf{q}^{(j)} - \mathbf{c}_t^{(j)}\|^2$$

Appendix A · Reference implementation

```
import numpy as np

def pq_encode(v, codebooks):
    # codebooks[j]: (k, D/m)
    subs = np.split(v, len(codebooks))
    return tuple(int(np.argmin(((cb - s)**2).sum(1)))
                 for s, cb in zip(subs, codebooks))

# 8-D vector, m=2 subspaces, tiny k=4 codebooks
v = np.array([1,2,0.5,1.5, 3,1,2,0])
cb1 = np.array([[1,2,0,1],[0,0,1,1],[1,2,0.5,1.5],[2,1,1,0]])
cb2 = np.array([[3,1,2,0],[0,0,0,0],[1,1,1,1],[2,2,1,0]])
pq_encode(v, [cb1, cb2]) # (2, 0)
# memory: 8*32 = 256 bits -> 2*log2(256) = 16 bits (16x, k=256)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0